



Full Stack Development Week 02

Topics Covered:

- Installation and setup
 - First React project
 - JSX
 - Recap
-

Activity 1: Check Node.js Installation

The first thing to check is the Node.js installation on your machine. Follow these instructions:

Check the Installation

Windows:

1. Open command prompt as administrator (search for it, right-click, and run as administrator).
2. Run these commands one after another:

```
node -v  
npm -v
```

3. If these commands are not recognized, install Node.js from scratch.
4. If versions are displayed, ensure they are the latest:
 - **Node (22.*)** (versions higher than 22 are fine)
 - **Npm (10.*)** (versions higher than 10 are fine)
5. If not, update your Node.js installation.

Mac:

1. Open Terminal.
2. Run these commands one after another:

```
node -v  
npm -v
```

3. If these commands are not recognized, install Node.js from scratch.
4. If versions are displayed, ensure they are the latest:
 - **Node (21.*)**
 - **Npm (10.*)**

5. If not, update your Node.js installation.

How to Install?

Windows:

1. Download the 32- or 64-bit version (depending on your machine) **CURRENT** version via MSI file from [Node.js Download](#).
2. Check versions again in command prompt.

Mac:

1. Download the macOS installer (.pkg) **CURRENT** version from [Node.js Download](#).
2. Check versions again in the terminal.

How to Update?

Windows:

- The easiest way is to reinstall as indicated above.

Mac:

1. Open terminal.
2. Install the `n` package via npm:

```
sudo npm install -g n
```

3. Update Node.js to the current version:

```
sudo n latest
```

Time to Ponder:

Why are Node and npm needed for React development?

Activity 2: Create First Next.js App

Next.js provides a powerful framework for building React applications with features like server-side rendering, static site generation, and API routes built-in.

1. Open command prompt or terminal.
2. Run the following command:

```
npx create-next-app@latest my-app
```

3. When prompted, select the following options:
 - Would you like to use TypeScript? → Yes
 - Would you like to use ESLint? → Yes
 - Would you like to use Tailwind CSS? → Yes
 - Would you like to use `src/` directory? → Yes
 - Would you like to use App Router? → No
 - Would you like to customize the default import alias? → No
4. Once completed, run:

```
cd my-app  
npm run dev
```

5. Open `http://localhost:3000` in your browser. If you see the Next.js welcome page, the setup is complete.
-

Activity 3: Study the Project Folder & Visual Studio Code

Spend some time reviewing the project folder created above.

Download **Visual Studio Code** (preferred IDE for this course) from [VS Code Download](#).

Note: You may use other IDEs, but support will only be available for VS Code.

Folder Structure Overview:

- **README.md**: A markdown file containing project details.
 - **node_modules/**: Contains all installed node packages. Do not modify this folder.
 - **package.json**: Lists project dependencies and configurations.
 - **package-lock.json**: Manages installed package versions.
 - **.gitignore**: Specifies files and folders that shouldn't be committed to git (e.g., **node_modules/**).
 - **public/**: Contains static assets like images that are served directly.
 - **src/**: Contains the application source code
 - **pages/**: Contains all page components and routing structure
 - **index.tsx**: The home page component (/)
 - **_app.tsx**: The root component wrapper
 - **styles/**: Contains CSS/styling files
 - **components/**: Contains reusable React components
 - **.eslintrc.json**: ESLint configuration for code linting
 - **tsconfig.json**: TypeScript configuration
 - **next.config.js**: Next.js configuration file
 - **postcss.config.js**: PostCSS configuration for Tailwind
 - **tailwind.config.js**: Tailwind CSS configuration
-

Activity 4: Typescript

TypeScript is a strongly-typed programming language that builds on JavaScript. It adds optional static types, classes, and modules to JavaScript, making it easier to develop large-scale applications. TypeScript code compiles into clean JavaScript code that runs in any browser or Node.js environment.

Key benefits of TypeScript include:

- Early error detection through static type checking
- Better IDE support with intelligent code completion
- Improved code maintainability and readability
- Enhanced debugging experience

Let's try some hands-on exercises using the [TypeScript Playground](#):

Exercise 1: Basic Type Annotations

Try these examples in the TypeScript Playground:

```
// Example 1: Basic Types
let name: string = "John";
let age: number = 25;
let isStudent: boolean = true;
let hobbies: string[] = ["reading", "gaming", "coding"];

// Try changing the values to incorrect types and see the errors
```

Exercise 2: Interfaces

```
// Define an interface for a Person
interface Person {
  name: string;
  age: number;
  email?: string; // Optional property
}

// Create objects using the interface
const student: Person = {
  name: "Alice",
  age: 20,
  email: "alice@example.com",
};

const teacher: Person = {
  name: "Bob",
  age: 35,
  // email is optional, so we can omit it
};
```

Exercise 3: Functions with Types

```
// Function with type annotations
function calculateArea(width: number, height: number): number {
    return width * height;
}

// Arrow function with type annotations
const greet = (name: string): string => {
    return `Hello, ${name}!`;
};

// Try calling these functions with different types of arguments
// and observe TypeScript's error messages
```

Try modifying these examples in the playground:

1. Change the types of variables and see what errors occur
2. Add new properties to the Person interface
3. Call the functions with incorrect parameter types

These exercises will help you understand how TypeScript's type system works and how it can catch errors before runtime.

Activity 5: JSX

- **What is JSX?**
- **What advantages does it offer?**
- **Provide an example involving JSX syntax.**

Activity 6: Revise HTML, CSS & JavaScript (Self-Exercise)

In your free time, review basic HTML, CSS, and JavaScript tutorials. References can be found in Module - Week 1 learning material:

- **HTML:** [HTML Tutorial](#)
- **CSS:** [CSS Tutorial](#)
- **JavaScript:** [JavaScript Tutorial](#)

It is crucial to be comfortable with these topics, as basic concepts will not be covered in this elective.